

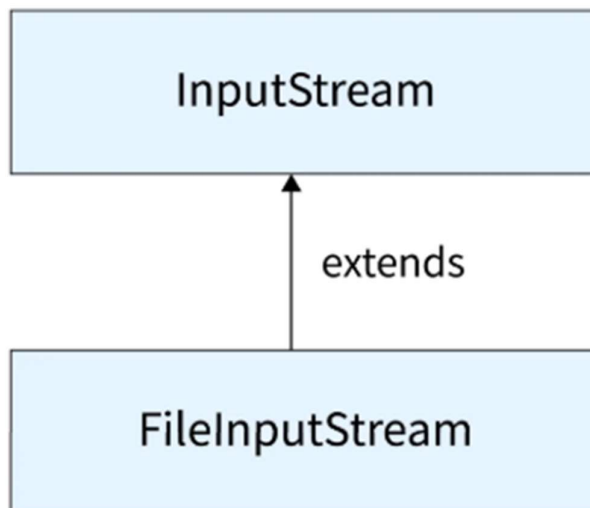
File Handling

✚ What is Java FileInputStream Class :-

- **FileInputStream** is used to **read data from a file** in the form of **bytes**.
- It is useful for reading **binary files** like images, audio, or video.
- It reads **raw byte data** (not text).
- For reading **text files**, use **FileReader** instead.
- It belongs to the **java.io** package.

✚ Syntax of FileInputStream :-

```
FileInputStream fis = new FileInputStream("filename.txt");
```

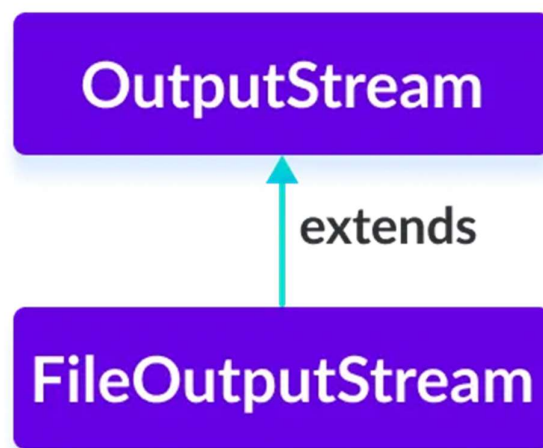


✚ What is Java FileOutputStream Class :-

- **FileOutputStream** is used to **write data to a file in bytes.**
- It is **byte-oriented**, suitable for **binary files** like images or audio.
- Belongs to the **java.io** package.
- **Extends** the OutputStream class.
- Can **create a new file** if it doesn't exist.
- Use write() method to **write data.**
- Always **close the stream** after writing.

✚ Syntax of FileOutputStream :-

```
FileOutputStream fos=new FileOutputStream("filename.txt");
```



Implementation of FileInputStream and FileOutputStream

Input :-

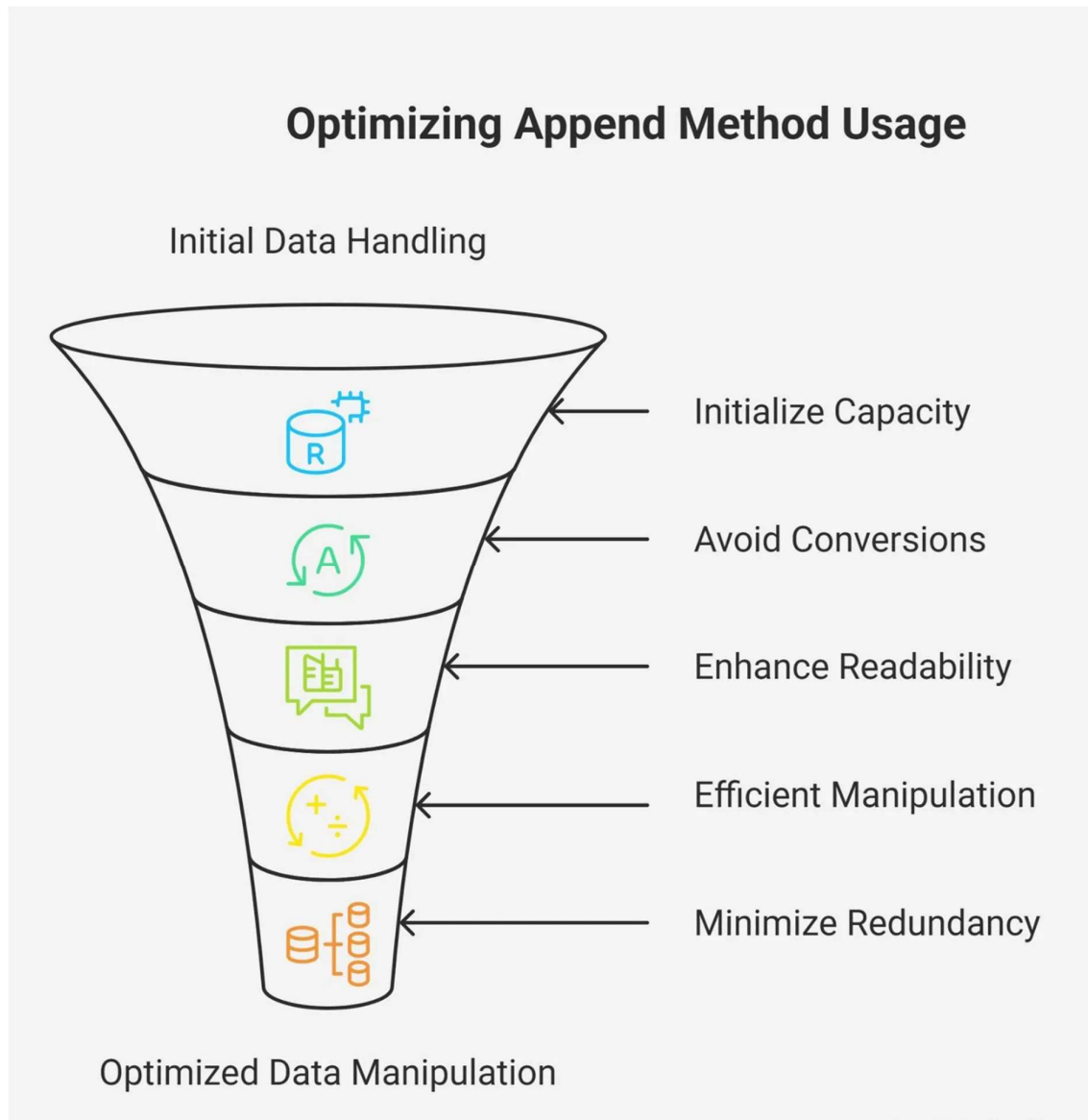
```
1 package file_handling;
2 import java.io.File;
3 import java.io.FileInputStream;
4 import java.io.FileOutputStream;
5 import java.io.IOException;
6
7 public class Filestreamclasses {
8
9     public static void main(String[] args) throws IOException {
10
11         File f=new File("C:\\Users\\HP\\OneDrive\\Desktop\\Fortune Cloud\\newfile.txt");
12
13         String name="Nikhil Shinde";
14
15         char ch='5';
16         FileOutputStream fos=new FileOutputStream(f);
17
18         fos.write(name.getBytes());
19         fos.write(ch);
20         fos.close();
21         System.out.println("File Written Successfully....!");
22
23         //Fir reading the file
24         System.out.println("Reading the file : ");
25         FileInputStream fis=new FileInputStream(f);
26         int i;
27         while((i=fis.read())!=-1)
28         {
29             System.out.print((char)i);
30         }
31     }
32 }
```

Output :-

```
File Written Successfully....!
Reading the file :
Nikhil Shinde5
```

Append Class :-

- Appending means **adding data to the end** of the file.
- Used with **FileOutputStream** by passing a boolean true in the constructor.
- false (default) — **overwrites** the file.
- true — **appends** to the file.



Implementation of append class :-

Input :-

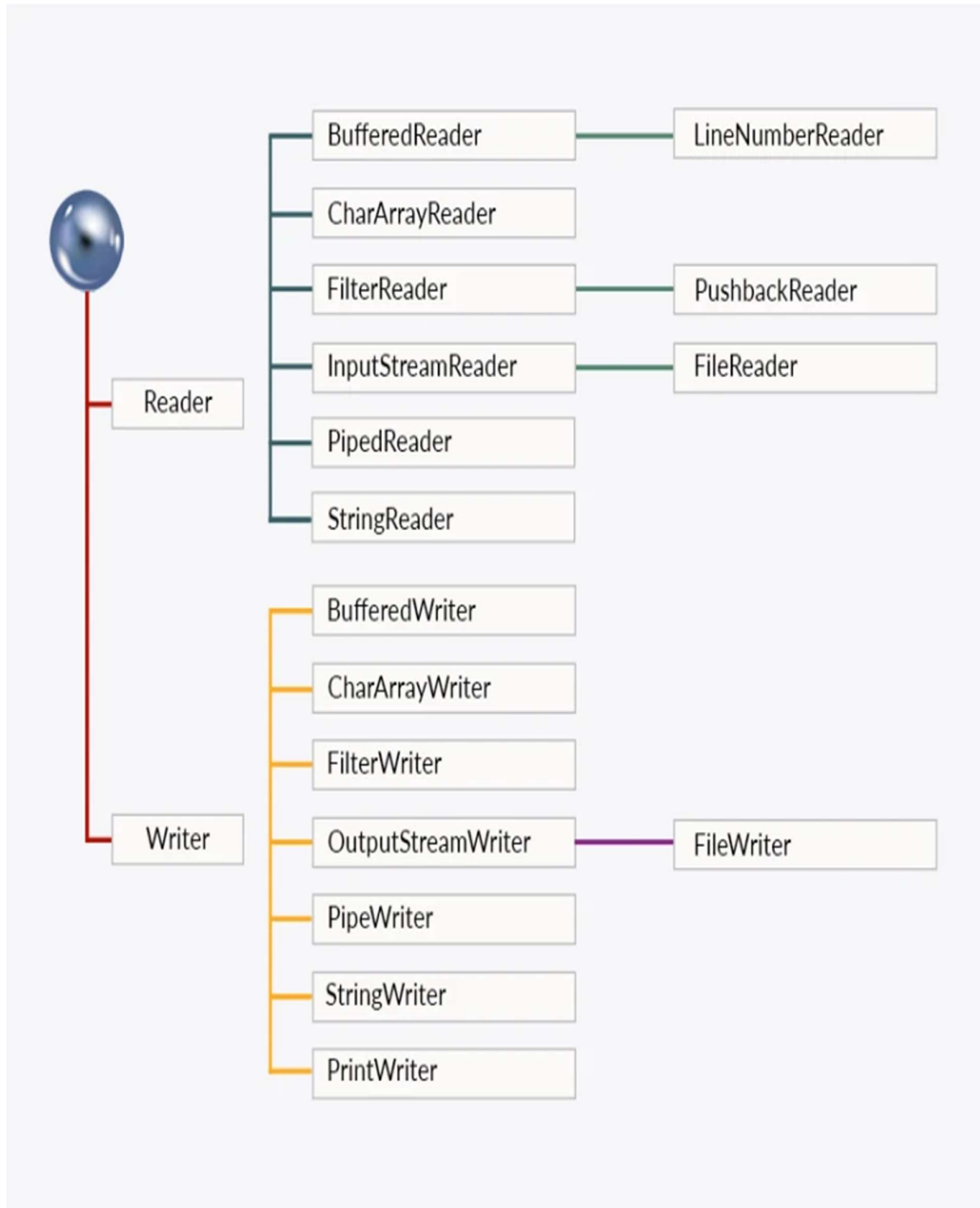
```
1 package file_handling;
2
3 import java.io.File;
4 import java.io.FileWriter;
5 import java.io.IOException;
6
7 public class append {
8
9     public static void main(String[] args) throws IOException {
10
11         File f=new File("C:\\\\Users\\\\HP\\\\OneDrive\\\\Desktop\\\\Fortune Cloud\\\\newfile.txt");
12         FileWriter fw=new FileWriter(f,true);
13         fw.write(" Pune");
14         fw.close();
15         System.out.println("File Written successfully....!");
16     }
17 }
```

Output :-

File Written successfully....!

✚ Bufferedreader and Bufferedwriter in java :-

➤ Reader and Writer Methods :-



Implementation of BufferedReader :-

Input :-

```
1 package file_handling;
2 import java.io.*;
3
4 public class buffer_reader {
5     public static void main(String[] args) throws Exception {
6
7         BufferedReader br = new BufferedReader(new FileReader
8             ("C:\\Users\\HP\\OneDrive\\Desktop\\Fortune Cloud\\newfile.txt"));
9
10        String line;
11        while ((line = br.readLine()) != null)
12        {
13            System.out.println(line);
14        }
15
16        br.close();
17    }
18 }
```

Output :-

Nikhil Shinde5 Pune

Implementation of Bufferedwriter :-

Input :-

```
1 package file_handling;
2
3 import java.io.BufferedWriter;
4 import java.io.FileWriter;
5
6 public class buffered_writer {
7
8     public static void main(String[] args) throws Exception {
9
10         BufferedWriter bw = new BufferedWriter(new FileWriter
11             ("C:\\Users\\HP\\OneDrive\\Desktop\\Fortune Cloud\\newfile.txt"));
12
13         bw.write("Hello, Java...!");
14         bw.newLine();
15         bw.write("Buffer Writer Example");
16         bw.close();
17     }
18 }
```

Output :-

```
Hello, Java...!
Buffer Writer Example
```


Difference between Bufferedreader and Bufferedwriter

Feature	BufferedReader	BufferedWriter
Purpose	To read text from a character stream efficiently	To write text to a character stream efficiently
Reads/Writes	Reads characters, lines, or arrays	Writes characters, lines, or arrays
Buffering	Uses a buffer to reduce I/O operations while reading	Uses a buffer to reduce I/O operations while writing
Common Methods	read(), readLine(), close()	write(), newLine(), flush(), close()
Typical Usage	new BufferedReader(new FileReader("file.txt"))	new BufferedWriter(new FileWriter("file.txt"))
Data Type	Reads text only (characters)	Writes text only (characters)
Performance	Faster than FileReader for reading large files	Faster than FileWriter for writing large files